

UNIVERSITÀ DEGLI STUDI DI VERONA

CORSO DI LAUREA IN INFORMATICA

Ingegneria del Software

Federico Brutti
federico.brutti@studenti.univr.it

Chiederò al tutor di venire incontro alla mia demenza senile. - Pietro S.

Indice

1	Modelli di processo di sviluppo software	5
1.1	Studio dei processi	5
1.2	Modelli a cascata	5
1.3	Modelli evolutivi	7
1.4	Ulteriori approcci	9
2	Ingegneria dei requisiti	13
2.1	Definizione e impatto dei requisiti	13
2.2	Processi di ingegneria dei requisiti	15
2.3	Use case	16
3	Progettazione del software	20
3.1	Pattern architetturali	20
3.2	Modelli di controllo	24
3.3	Principi di progettazione	26
3.4	Design Pattern	29
4	UML - Unified Modeling Language	33
4.1	Introduzione a UML	33
4.2	Diagrammi strutturali	35
4.3	Diagrammi di interazione	38
4.4	State Machine Diagram	40
4.5	Activity Diagram	41
4.6	Component, Package e Deployment Diagram	43
5	Evoluzione e mantenimento software	44
5.1	Refactoring	44
5.2	Unit testing	44

Introduzione

L'**ingegneria del software** è una disciplina ingegneristica riguardante ogni aspetto relativo allo sviluppo del software professionale, dalla specifica del sistema fino alla manutenzione dopo che è entrato in uso. Aiuta a ottenere prodotti con la qualità richiesta entro i limiti di tempo e costo, accettando anche compromessi. Si tratta di un approccio sistematico e organizzato, orientato in base al caso di studio.

Esistono infatti degli insiemi di attività, detti **processi di sviluppo**, atti alla creazione di un prodotto software. Si dividono in due macrocategorie: **plan-driven**, dove lo sviluppo è pianificato in modo dettagliato e le fasi sono seguite secondo un piano prestabilito, e **processi agili**, che garantiscono un notevole adattamento ai cambiamenti tramite un frequente confronto con il cliente. Nonostante la diversità, i processi di sviluppo condividono quattro attività fondamentali:

- **Specifiche del software:** Clienti e ingegneri definiscono funzionalità e vincoli del software.
- **Sviluppo del software:** Progettazione e implementazione del sistema.
- **Convalida del software:** Testing del software per garantire che rispetti le richieste del cliente.
- **Evoluzione del software:** Il prodotto viene modificato per soddisfare eventuali cambiamenti dei requisiti del cliente e del mercato.

Non esiste il processo perfetto e universalmente applicabile ad ogni caso di studio, ragion per cui nei prossimi capitoli studieremo diverse tecniche, le quali si potranno scegliere in base ai quattro aspetti più influenti per lo sviluppo:

- **Diversità:** Necessità che il software si adatti a diverse piattaforme o tecnologie.
- **Variabilità:** È verosimile che con il cambiare del mercato, i committenti modifichino a loro volta i requisiti. La sfida della variabilità consiste nell'essere agili abbastanza da sviluppare e modificare il software per adattarlo a queste casistiche.

- **Fiducia e protezione:** Essendo il software radicato in ogni aspetto della vita è necessario che funzioni correttamente e non sia vulnerabile ad attacchi di malintenzionati.
- **Scala:** Il software deve essere sviluppato per sistemi di dimensioni molto diverse, dai piccoli dispositivi fino a grandi sistemi distribuiti e servizi cloud.

Come ci si può aspettare, lo sviluppo di software professionale di grandi dimensioni è infattibile da un singolo individuo: per questa ragione, in condizioni ideali, le aziende creano una task-force per la gestione del lavoro, composta dalle seguenti figure:

- **Analista:** Definisce i requisiti decisi insieme al committente.
- **Designer:** Si occupa della strutturazione di design ad alto e basso livello.
- **Programmatore:** Scrive il codice.
- **Tester:** Verifica la correttezza del codice.
- **Mantenitore:** Si occupa della manutenzione del software dopo il rilascio, correggendo errori, migliorando il sistema e adattandolo a nuovi requisiti o tecnologie.
- **Project Manager:** Responsabile della pianificazione e controllo di un progetto. Gestisce inoltre il personale, stima i costi e cerca di far rispettare budget e tempi. Idealmente saprebbe giostrarsi bene in ognuno dei campi precedenti.

Non sempre lo sviluppo del software va come pianificato; possono presentarsi problemi di vario tipo, come per esempio sforare il budget iniziale o i tempi di consegna. Ciò può accadere per una moltitudine di motivi, ma da vari report emerge che le cause principali derivano da una comunicazione non esaustiva tra **stakeholder** e team operativo, dalla difficoltà nel comprendere il dominio di lavoro, oppure da problemi organizzativi nel team.

Come vedremo nei seguenti capitoli, sarà di fondamentale importanza definire appropriatamente i requisiti e useremo vari sottoprocessi per "estrarre" quante più richieste possibile dai committenti. In sistemi normali, la loro validazione avviene tra analista e stakeholder, ma esistono software il cui funzionamento è fondamentale che sia garantito. Parliamo in questo caso di **sistemi critici**, come macchinari utilizzati in ambito medico, per i quali il software deve rispettare standard rigorosi ed essere certificato da enti di validazione indipendenti.

Capitolo 1

Modelli di processo di sviluppo software

1.1 Studio dei processi

Come già detto, un **processo software** è un insieme di attività atte alla creazione di un software, comprende almeno le quattro attività fondamentali e si sceglie in base al caso di studio. I vantaggi di una sua definizione sono evidenti: garantire uno standard lavorativo e di conseguenza un livello minimo di qualità del prodotto. Esistono numerosi processi software, dai più **prescrittivi**, caratterizzati da regole e fasi ben definite, ai più **adattivi**, che privilegiano flessibilità e adattamento ai cambiamenti.

Conoscere i modelli di processo è fondamentale per i **project manager**, poiché è compito loro decidere quale adottare considerando la tipologia del software da progettare. È un tema complesso, perché ha un enorme impatto sulla buona riuscita del progetto. Un secondo motivo per cui è bene conoscere i processi è uscire da quello di bruteforcing, conosciuto anche come **code and fix**, verosimilmente utilizzato da te fino all'avvento di questa materia. Si tratta di un *modus operandi* privo di analisi e progettazioni, il cui prodotto si crea esclusivamente per tentativi. È chiaro che non è un metodo adatto per progetti professionali, quindi nelle prossime sezioni analizzeremo le tipologie di processo più usate: a cascata, evolutivi, basati sulla configurazione e integrazione di componenti e agili.

1.2 Modelli a cascata

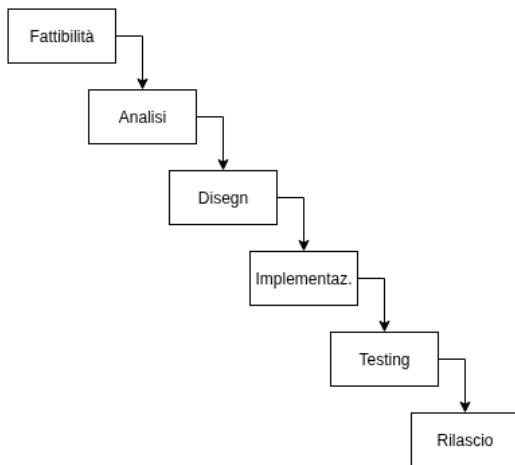
Storicamente, il modello **Waterfall** è il primo mai utilizzato in ambito di sviluppo software. Mette in cascata i passi tradizionali dello sviluppo di un sistema ed ogni fase produce un **deliverable**, il quale verrà usato come input di quella successiva, fino al termine del processo. Presenta i seguenti passi:

- **Studio della fattibilità:** Valutazione di fattibilità e convenienza in base al tempo e le risorse disponibili. Si stima inoltre il **return on investment** (ROI), ovvero il guadagno potenziale derivante dall'investimento nello sviluppo del software.
- **Definizione e analisi dei requisiti:** Raccolta delle richieste del cliente. Il deliverable è il **documento dei requisiti**, il quale fornisce i dettagli in merito alle funzionalità del programma. Può avere definizioni, specifiche o visualizzazioni grafiche.
- **Design del software:** La definizione delle componenti del sistema con le relative relazioni (**high-level design**) oppure la definizione interna di ogni componente (**low-level design**). Il deliverable è la specifica del design mediante un **linguaggio di modellazione unificato** (UML).
- **Implementazione del codice:** La scrittura effettiva del codice insieme al mandatorio unit testing. Ci sono diversi paradigmi di sviluppo, ma il deliverable rimane sempre il codice completo del programma.
- **Verifica del codice:** Il testing del codice con l'eventuale integrazione di soluzioni ai problemi. Il deliverable fornisce i dettagli sui test effettuati insieme ad un report.
- **Mantenimento del codice:** La costante manutenzione del software per la durata del suo ciclo di vita, come l'aggiunta di nuove feature, ottimizzazioni o di adattare il codice alle nuove tecnologie, come l'ultima versione di un sistema operativo.

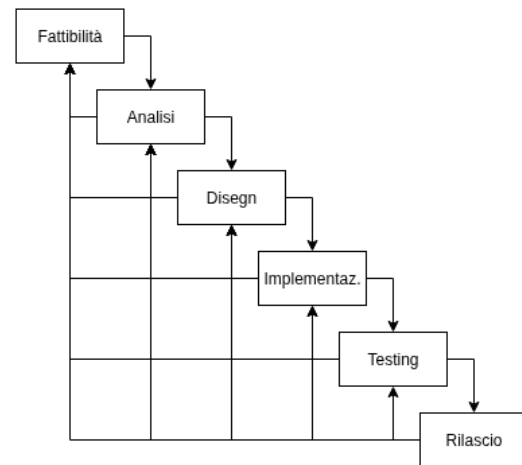
I pregi dell'uso di questo metodo sono l'enfasi sull'analisi dei requisiti ed il progetto di sistema, perché prima di cominciare ci si assicura di aver determinato correttamente i bisogni del cliente e di averli definiti adeguatamente. Sebbene questo sia un processo solido, allo stesso tempo risulta rigido e monolitico, rendendo difficile contemplare cambiamenti al progetto in corso d'opera.

Per far fronte a questo problema di flessibilità sono stati ideati altri modelli ispirati al waterfall che implementano il concetto di feedback e interazioni durante lo sviluppo. Per esempio, nel **modello a cascata iterativo** si ha la possibilità di ritornare a una fase precedente in base al feedback dato dallo stakeholder, ottenuto di norma fornendo un prototipo. Un'altra variante è il **modello a V**, che dà molta enfasi al testing, mostrando anche come le loro attività siano collegate a quelle di analisi e progettazione. In questo paradigma, infatti, ogni controllo che non ha esito positivo porta a un rifacimento o modifica della fase associata, per poi riprendere il workflow da quella fase.

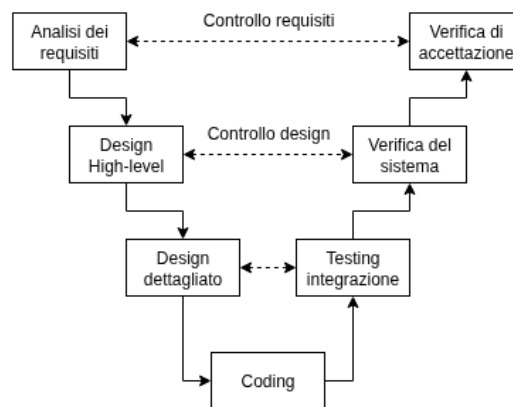
Sebbene questi due modelli abbiano apportato migliorie al waterfall, è evidente che siano ugualmente poco flessibili agli imprevisti. Nella maggior parte dei casi non è realistico che i requisiti rimangano gli stessi per quelli che potrebbero essere anni di lavoro. Per questo motivo sono stati ideati altri paradigmi di sviluppo software con alla base la possibilità di evolversi in base al feedback del cliente.



(a) Modello a cascata



(b) Modello a cascata iterativo



(c) Modello a V

1.3 Modelli evolutivi

I **modelli evolutivi** si basano sull'idea di sviluppare una versione iniziale del software, incompleta ma funzionante, da sottoporre agli utenti e raffinare progressivamente tramite feedback, fino al raggiungimento delle funzionalità richieste.

Un primo esempio è dato dai **modelli di prototipizzazione**, la cui implementazione è data da prototipi del sistema, sui quali validare i requisiti. Se questo soddisfa le richieste degli utenti, può essere evoluto nel sistema finale; in alternativa, può essere scartato dopo aver chiarito i requisiti. Essendo spesso un artefatto temporaneo, il prototipo non viene sviluppato con particolare attenzione all'efficienza o alla qualità del codice; infatti sono usati strumenti e tecnologie per una rapida realizzazione.

Questo modello di sviluppo consente di raffinare al meglio i requisiti definiti dagli stakeholder e permette di rilevare errori di interpretazione molto facilmente. Tuttavia, si

tratta di un processo dispendioso in termini di risorse fisiche e psichiche. In particolare, esiste il rischio di considerare il prototipo come prodotto finale, compromettendo la qualità del sistema e annullando i benefici della prototipizzazione. Per questa ragione i prototipi riguardano tendenzialmente funzionalità su cui il cliente ha fornito requisiti ambigui.

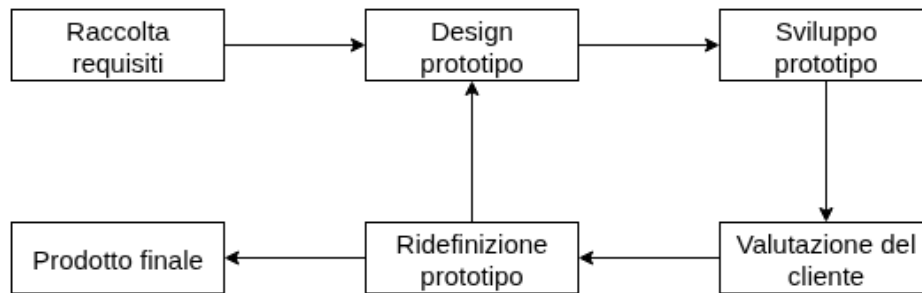


Figura 1.2: Modello di prototipizzazione

Due altri modelli che consentono una rapida risposta alle richieste degli stakeholder e del mercato sono quelli **incrementali** e **iterativi**, che prevedono lo sviluppo di più release utilizzabili di cui solo l'ultima è completa. In particolare, per il primo si richiede di individuare le funzionalità fondamentali del software per il primo rilascio (qui si implementano tipicamente i requisiti a maggior valore per gli utenti), al quale saranno aggiunte altre funzioni con l'avvento delle successive release, fornendo versioni del software sempre più complete.

Per i modelli **iterativi**, invece, si sviluppa una versione iniziale del sistema che viene progressivamente migliorata e raffinata a ogni iterazione, sia in termini di funzionalità sia di qualità. Per esempio, si potrebbe sviluppare un word processor funzionante con una CLI, per poi fare una seconda release con una GUI.

I vantaggi di questi modelli risiedono nella possibilità di ottenere rapidamente versioni funzionanti del software, raccogliere feedback dagli stakeholder e adattare lo sviluppo di conseguenza, riducendo il rischio di errori nei requisiti e migliorando progressivamente il prodotto.

Il principale modello di sviluppo evolutivo è il **modello a spirale**: si tratta di un approccio realistico per sistemi di grandi dimensioni, il quale si evolve insieme alle interazioni continue tra stakeholder e team operativo. È un workflow guidato dal rischio, inteso come una circostanza potenzialmente avversa in grado di compromettere lo sviluppo e la qualità del software.

Nel lavoro è necessario fare delle scelte, e, come per gli investimenti, comportano un rischio. Per questa ragione, bisogna valutare la gravità delle conseguenze e la probabilità

che la circostanza si verifichi. Il modello presenta quattro fasi entro le quali si itera fino al raggiungimento del prodotto definitivo:

- **Pianificazione:** Determinazione degli obiettivi, alternative e vincoli da rispettare.
- **Analisi dei rischi:** Identificazione dei rischi, alternative ed eventuali risoluzioni.
- **Sviluppo del software:** Sviluppo del prodotto.
- **Convalida del cliente:** Revisione dei risultati da parte del committente.

I principali vantaggi risiedono nella grande flessibilità ai problemi, ma non è una soluzione universale e richiede una buona competenza nella valutazione dei rischi, perché in base alle conseguenze, potrebbe essere necessaria una significativa revisione del prodotto.

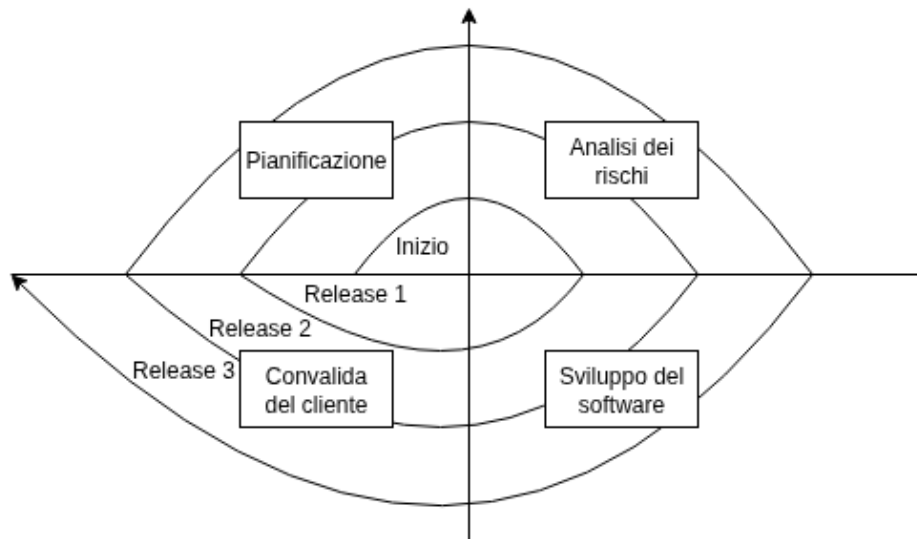


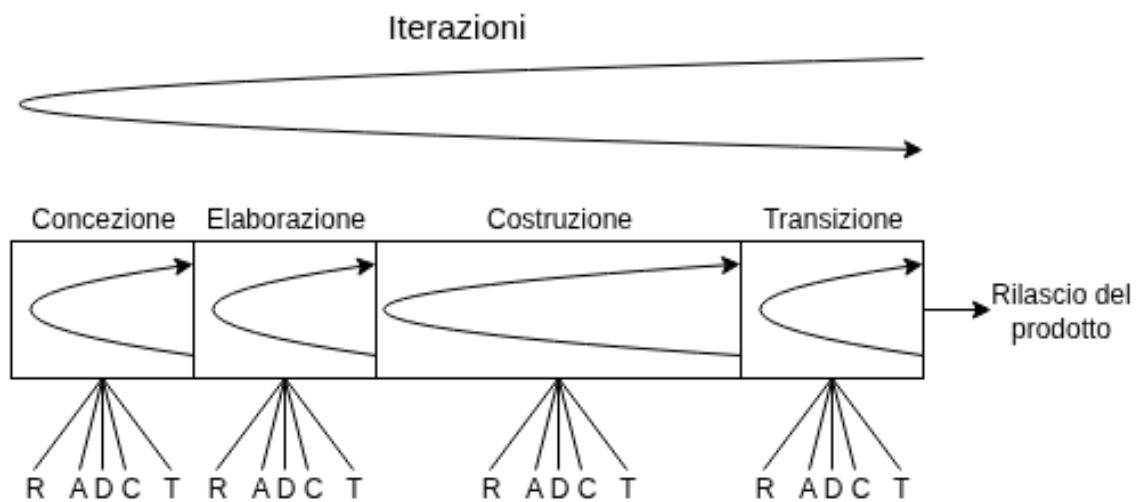
Figura 1.3: Modello a spirale

1.4 Ulteriori approcci

I modelli di sviluppo non si limitano solo ad essere guidati da un piano oppure dal feedback del cliente, bensì possono integrare idee di entrambi, come per l'**unified process**: un approccio per lo sviluppo di sistemi a oggetti con notazioni UML. È guidato dagli **use case** e incorpora alcune idee del modello a spirale, come i rilasci multipli dopo ogni fase e una forte enfasi sulla convalida del cliente, mantenendo però una struttura plan-driven. Il modello presenta quattro fasi:

- **Concezione:** Studio di fattibilità e definizione dei requisiti essenziali.
- **Elaborazione:** Studio del dominio applicativo e definizione dell'architettura del software.
- **Costruzione:** Design del prodotto, codifica e testing.
- **Transizione:** Utilizzo del software nel contesto applicativo da parte di utenti fidati.

Ognuna di queste fasi presenta cinque workflow: **Requisiti, Analisi, Design, Codifica e Testing**.



Un altro approccio con una diversa filosofia è dato dallo **sviluppo basato sui componenti**, la cui idea fondamentale è la minimizzazione di riscrittura di nuovo codice, riutilizzando componenti già a disposizione, piuttosto che scriverne di nuovi. Idee come questa sono supportate dalla presenza di un mercato degli asset, dal quale si attinge. Presenta le seguenti fasi:

- **Specifica dei requisiti**
- **Analisi dei componenti**
- **Modifica delle richieste**
- **Design orientato al riutilizzo**
- **Sviluppo e integrazione**

- **Convalida del sistema**

Lo sviluppo inizia dunque dalla ricerca dei componenti in base alle richieste del cliente; questi sono forniti dal marketplace, aziende terze o da progetti open-source. Eventualmente, se non rispettano i requisiti, è possibile ridefinirli con il committente oppure cercare di sostituirli; dopodiché si procede con l'implementazione ed infine la validazione.

Il vantaggio principale è il risparmio di tempo: affidandoci a prodotti già scritti e funzionanti si riducono costi totali e rischi. Tuttavia, non è sempre facile allineare i desideri del committente con le componenti a disposizione, spesso non modificabili. Inoltre, essendo dipendenti da terzi, le parti utilizzate potrebbero venire aggiornate al di fuori dal controllo del team.

Esistono infine modelli basati su metodi formali, dove si richiede che venga tutto definito in modo rigoroso e con prove matematiche, come il **modello trasformativale**. La specifica viene formalizzata e guida lo sviluppo fino ad arrivare all'implementazione. Risulta vantaggioso perché ci consente di provare con (quasi) assoluta certezza il funzionamento del codice grazie alle relative dimostrazioni.

Presentiamo qualche cenno sul workflow più utilizzato in ambito lavorativo odierno: i **metodi agili**. Qui si pone meno enfasi sulla documentazione formale, privilegiando la comunicazione diretta e la produzione di software funzionante per garantire una maggiore flessibilità ai cambi repentini del mercato e degli stakeholder. La filosofia di questi modelli pone la soddisfazione del cliente come obiettivo più importante, rispetto a seguire un piano. Presentano le seguenti caratteristiche:

- **Ciclo rapido e consegne frequenti:** Ogni iterazione produce un incremento funzionante del software, rilasciato in tempi brevi.
- **Progettazione semplice:** Sviluppo orientato alla realizzazione delle funzionalità essenziali, evitando complessità non necessarie.
- **Refactoring:** Il codice è costantemente ristrutturato per renderlo quanto più chiaro possibile. Il codice dovrebbe quindi "commentarsi da solo".
- **Meeting di riflessione:** Incontri periodici per pianificare il lavoro, monitorare l'avanzamento e riflettere sui miglioramenti del processo.
- **Conoscenza tacita e implicita:** Si privilegia lo scambio di informazioni tra i membri del team rispetto a documentazione formale eccessiva. (Pratica dubbia, ma efficiente sul breve termine).
- **Co-Location:** Stretta interazione con gli stakeholder, idealmente con una presenza costante o facilmente accessibile.

È evidente che questi metodi presentano alcune criticità, tra cui una maggiore dipendenza dalla collaborazione tra i membri del team, difficoltà nella manutenzione in caso di documentazione insufficiente e da un punto di vista legale, data la flessibilità (e quindi ambiguità) del contratto.

Capitolo 2

Ingegneria dei requisiti

2.1 Definizione e impatto dei requisiti

I **requisiti** di un sistema sono la descrizione dei suoi servizi e vincoli operativi; il processo della loro ricerca, analisi, documentazione e verifica è detto **ingegneria dei requisiti**. Queste caratteristiche riflettono le richieste del cliente e sono quindi orientate alla risoluzione dei suoi problemi.

Si tratta di una fase fondamentale del processo lavorativo, perché l'incompletezza o ambiguità dei requisiti è uno dei fattori maggiori che portano al fallimento di un progetto oppure alla sua rielaborazione, specie se in fasi avanzate dello sviluppo, con costi molto elevati. In ogni caso, definiamo due tipologie di requisiti:

- **Utente:** Descrizione in linguaggio naturale dei vincoli operativi e servizi forniti dal sistema. Spesso accompagnati da diagrammi per aumentarne la comprensibilità.
- **Di sistema:** Descrizione dettagliata e formale¹ di funzioni, servizi e vincoli del sistema. Comprime il **documento dei requisiti**, il quale specifica esattamente che cosa deve essere implementato. La sua scrittura può essere parte del contratto tra stakeholder e azienda.

È buona prassi scrivere i requisiti in più livelli di dettaglio, poiché diversi tipi di lettori li usano in modo differente. Per esempio, un receptionist che si occupa del concierge in hotel avrà bisogni differenti rispetto al manager.

Per scrivere un documento dei requisiti si possono usare strumenti come diagrammi UML, testi strutturati e use case. Nei metodi agili, invece, si fa spesso uso delle user stories.

¹Si indicano il nome della funzione, una sua descrizione, gli input, la sorgente, gli output, la destinazione, i parametri, pre-condizioni, post-condizioni, effetti collaterali e il link al requisito utente.

Esempio 1. Requisiti utente

La seguente lista puntata indica i requisiti di un software in linguaggio naturale e comprensibile per tutti:

Aggiungere nodi a un design

1. *L'editor fornisce un mezzo per aggiungere nodi di un tipo specifico al design.*
2. *La sequenza di azioni per aggiungere un nodo deve essere:*
 - (a) *L'utente seleziona il tipo di nodo da aggiungere.*
 - (b) *L'utente muove il cursore sulla posizione del nodo approssimata nel diagramma e indicare che il simbolo del nodo dovrebbe essere aggiunto a quel punto.*
 - (c) *L'utente trascina il simbolo del nodo alla posizione finale.*

I requisiti dei sistemi software sono spesso divisi anche in queste due categorie:

- **Requisiti funzionali:** Definizioni di servizi che il sistema deve fornire; indicano come reagisce a determinati input e come dovrebbe comportarsi nelle varie situazioni.
- **Requisiti non funzionali:** Vincoli su funzioni e servizi del sistema, come prestazioni, sicurezza, affidabilità o vincoli tecnologici. Includono inoltre limitazioni su tempo, processo di sviluppo e quelle richieste dagli standard. Di norma si applicano al sistema in toto e non a singole funzioni.

Esempio 2. Requisiti funzionali e non funzionali

Si consideri un sistema bancomat. I requisiti non funzionali indicano richieste indipendenti dalle funzionalità:

- *Il sistema deve essere facilmente espandibile e adattabile alle future esigenze bancarie.*
- *Il sistema deve essere disponibile a persone portatori di Handicap, deve garantire un tempo di risposta inferiore al minuto e deve essere sviluppato su architettura X86 con sistema operativo compatibile con quello della banca.*

Mentre i requisiti funzionali riguardano i servizi, anche se scritti in maniera generale:

- *Il sistema deve mettere a disposizione le funzioni di prelievo, saldo, estratto conto.*
- *Le operazioni di prelievo devono richiedere autenticazione tramite un codice segreto memorizzato sulla carta.*

2.2 Processi di ingegneria dei requisiti

Come già menzionato, il fine ultimo dell'ingegneria dei requisiti è il documento relativo alle funzionalità e servizi offerti dal sistema. L'attività è divisa in quattro fasi chiave:

- **Elicitazione dei requisiti:** Ingegneri e stakeholder lavorano insieme per studiare il dominio applicativo e definire le funzioni del sistema.

Si utilizzano diverse tecniche, come le interviste, i questionari, i workshop, oppure approcci più diretti come l'osservazione sul posto di lavoro o l'analisi dei prodotti dei competitor.

- **Analisi dei requisiti:** Si raffinano i requisiti raccolti e si stabilisce la loro priorità², fattibilità e correttezza. In presenza di lacune o ambiguità, si cerca di colmarle ricavando altri requisiti o chiedendo ulteriori spiegazioni agli stakeholder.

Può capitare inoltre di avere conflitti tra i requisiti; la risoluzione risiede nella negoziazione con i committenti per una modifica o rimozione della richiesta.

- **Specificazione dei requisiti:** Processo di scrittura dei requisiti utente e di sistema in due documenti; il primo è redatto per i clienti e costituisce un contratto fra le parti, mentre il secondo è scritto per gli sviluppatori in un linguaggio tecnico e formale. Quest'ultimo rappresenta il punto di partenza per la fase di design del software.

- **Convalida dei requisiti:** Processo di verifica sui requisiti, per assicurarsi che descrivano il software realmente voluto dal cliente. Si cerca di sanare ogni ambiguità e confermare di avere un insieme completo per rispettare le richieste.

Normalmente la convalida avviene mediante revisioni tra pari, mirando ad identificare aree di sviluppo o miglioramento. Alternativamente è possibile usare dei test case oppure sviluppare un prototipo.

Naturalmente, nelle prime fasi del lavoro si spenderanno più sforzi sulla definizione dei requisiti non funzionali, sicché siano coerenti con i desideri degli stakeholder, mentre nelle ultime fasi, appurata la loro correttezza, si potrà dare più importanza ai requisiti funzionali, per guidare al meglio il lavoro di scrittura del codice.

Creato e validato il documento dei requisiti, è necessario mantenere una loro gestione, in quanto questi possono cambiare per dinamiche lavorative e di mercato; quando succede, è necessario richiedere una revisione dei requisiti con gli stakeholder.

²Discussione dei requisiti in termini di **must**, **could** e **should**, quindi secondo la scala MoSCoW

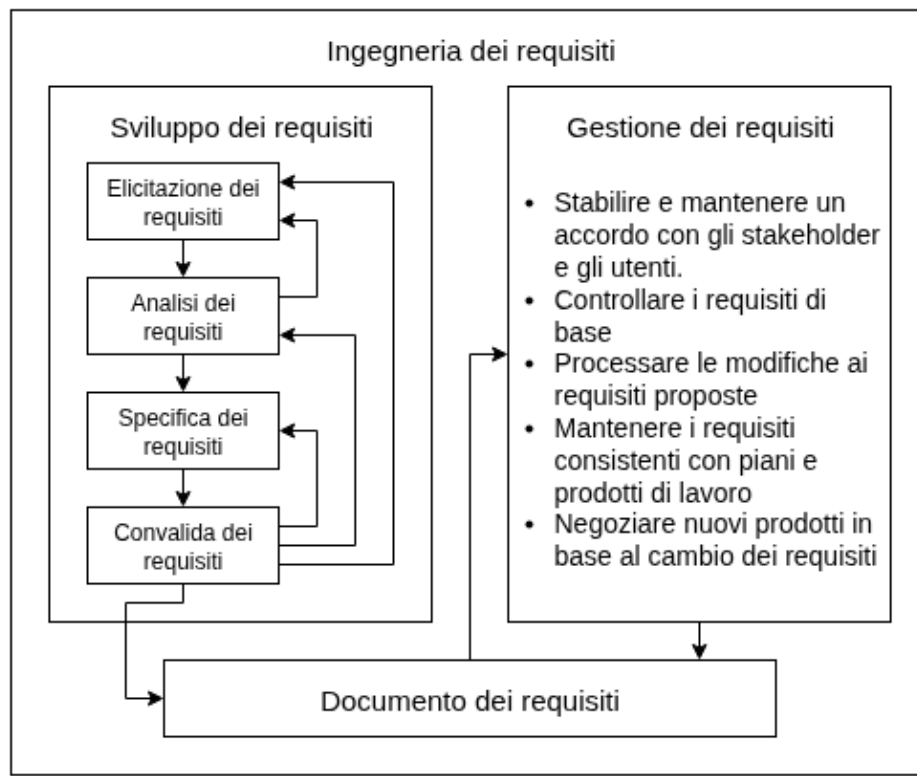


Figura 2.1: Processi dell'ingegneria dei requisiti

2.3 Use case

L'utilizzo di **use case** è una tecnica non visuale per l'espressione dei requisiti funzionali di un sistema e la descrizione del sistema dal punto di vista dell'utenza, senza che questo sia cosciente degli eventi che avvengono al suo interno. Il software viene infatti visto come una black box della quale si descrive esclusivamente lo scambio di messaggi tra il sistema e le entità attive, dette **attori**. Il caso d'uso può presentare una "storia" che descrive una modalità di utilizzo da parte dell'utente. Per esempio, il programma di un ATM può essere descritto in due passaggi:

1. L'utente seleziona l'importo da prelevare e conferma la cifra.
2. Il sistema eroga la cifra.

Formalmente, gli use case hanno una struttura specifica; un insieme di campi che comprendono in primo luogo l'**ID del caso**, il **nome del caso**, una breve **descrizione dell'obiettivo** e gli **attori**. In una sezione sottostante è poi spiegato in termini semplici il **workflow principale**, ovvero l'istanza di utilizzo del software ideale, insieme

ai **workflow secondari**, che mostrano il comportamento del sistema quando si devia dall'interazione ideale.

Principalmente, gli use case sono utili a documentare le scelte prese per rispettare i requisiti e presentare chiaramente il risultato agli stakeholder. Sono inoltre passati al team di sviluppo che progetterà l'applicativo, i quali potranno usarli come base per valutare la qualità del software.

Per dare una visione ulteriormente chiara degli use case, è possibile ricondursi ad una forma grafica: gli **use case diagram**, composti da:

- **Attori:** Ruoli assunti dalle entità che interagiscono con il sistema. Si dividono a loro volta in
 - **Primari:** Entità che hanno uno scopo raggiungibile grazie al sistema, come un utente che vuole comprare un prodotto su un sito di e-commerce.
 - **Secondari:** Entità che producono qualcosa per il sistema, come il circuito di pagamento per l'elaborazione delle transazioni.

Possono essere anche organizzati in gerarchie e se trattano utenti sono rappresentati da stickmen.

- **Use case:** Ciò che gli attori possono eseguire con il sistema. Rappresentati da ellissi.
- **Relazioni:** Legami tra attori e use case. Rappresentate da archi tra le ellissi.
- **Confini del sistema:** Definisce l'insieme delle operazioni che il sistema può effettuare. Rappresentati da un rettangolo che contiene gli elementi precedenti.

La definizione dei confini del sistema è in particolar modo importante, poiché ha un grosso impatto sui requisiti funzionali; molti progetti sono falliti esattamente per questo problema.

Altro concetto importante è quello di **scenario**; una sequenza ordinata di interazioni tra un sistema e gli attori che rappresenta una particolare istanza, e quindi un singolo cammino, di uno use case. Lavorare con degli scenari concreti è particolarmente utile perché possono essere utilizzati per il testing del codice. Naturalmente, un caso d'uso contiene più scenari e in questi termini possiamo formalmente definirli come insiemi di scenari che hanno in comune lo scopo finale dell'utente.

Prendiamo nuovamente in considerazione l'esempio dell'ATM; lo scenario ideale comprenderebbe:

1. L'utente esegue un tentativo di autenticazione.
2. Il sistema verifica la correttezza dei dati e conferma l'accesso.

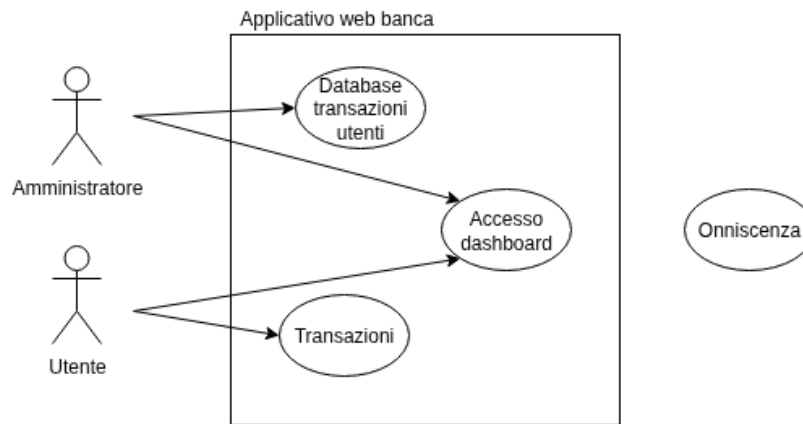


Figura 2.2: Esempio di use case diagram

3. L'utente inserisce l'importo da prelevare e conferma la cifra selezionata.
4. Il sistema controlla che l'importo sia disponibile, ed in tal caso eroga il contante.

Tuttavia, è possibile che si verifichino altri scenari secondari, come l'inserimento di un pin errato, oppure la richiesta di prelevare più soldi di quanti sono disponibili sul conto. Queste istanze sono dette **deviazioni** e devono essere appropriatamente gestite e documentate. Quando ritornano nel workflow principale sono dette **semplici**, altrimenti si dicono **complesse**.

Inoltre, per la descrizione degli scenari è necessario specificare anche le **pre-condizioni**, i vincoli sullo stato corrente del sistema, e le **post-condizioni**, le condizioni che devono risultare vere quando lo use case termina con successo.

Definiamo ora le possibili relazioni tra gli use case:

- **Inclusione:** Usato per associare un comportamento a più use case. Nella descrizione testuale si indica con `.include(caseName)`, mentre in quella grafica si scrive sopra l'arco `<< include >>`.
- **Estensione:** Usato per indicare un comportamento opzionale. Indicato via testo con `.extends(caseName)` e `<< extends >>` via grafica.
- **Generalizzazione:** Usato per la generalizzazione di più use case. Per esempio, se dei casi usano una stessa funzione di pagamento, si raggrupperanno in un altro use case più generale. Viceversa, si possono vedere i casi raggruppati come istanze più specifiche.

A supporto dei diagrammi degli use case è possibile usare dei tool appositi per fare degli **screen mockup**, ovvero degli schizzi dell'interfaccia utente per chiarirne il comportamento. Sono utili per aumentare la comprensibilità delle funzioni e a guidare lo sviluppo del software dando un'idea di come dovrebbe risultare. Un esempio di questi tool sono quelli per la **GUI prototyping**.

Riassumendo, è bene scrivere use case brevi e semplici, con un buon compromesso fra astrazione e dettagli per la descrizione del comportamento di un sistema. Ne esistono diversi template equivalenti e sono molto utilizzati in ambito lavorativo. L'ambito in cui possono risultare meno utili è quello di sistemi dominati da requisiti non funzionali o con algoritmi complessi il cui funzionamento non necessita accoppiamento con interfaccia grafica.

Capitolo 3

Progettazione del software

3.1 Pattern architetturali

La **progettazione architetturale** si occupa dell'organizzazione di un software e della definizione di una sua struttura complessiva; si tratta del collegamento fra ingegneria dei requisiti e la progettazione, poiché si identificano i componenti strutturali del sistema e le loro relazioni. Il deliverable è un modello architetturale che descrive in che modo il software è organizzato, in funzione dei suoi componenti.

Sono proprio le componenti a identificare le caratteristiche di alto livello del sistema, ragion per cui è data molta importanza alla loro definizione. Si potrebbe inoltre usare il modello risultante per discutere con gli stakeholder dei progressi fatti.

In questa fase di sviluppo stiamo assumendo che il documento dei requisiti sia completo e disponibile, perché verrà usato come base per il design del sistema. Conoscendo "cosa" il software deve fare, adesso dobbiamo definire "come" deve eseguirlo, quindi la **struttura** della soluzione, la quale verrà scritta nella fase di implementazione. In particolare, ci sono due livelli di astrazione per la progettazione di architetture software:

- **Alto livello:** Definizione dei macrocomponenti sui quali il sistema verrà architettato.
- **Basso livello:** Definizione delle singole componenti, specificando la soluzione nel dettaglio.

Quello della progettazione è un processo creativo, basato nella maggior parte dei casi sull'adattamento di strutture già conosciute al caso in esame. Generalmente si fa molto uso degli **stili architetturali** ad alto livello e dei **design pattern** a basso livello. La definizione dell'architettura del sistema ha svariati pregi, come fornire una **guida** allo sviluppo, semplificando la comprensione del software in ambito di scelte manageriali e

analisi di proprietà. Come conseguenza diretta, il sistema viene documentato e si ha una visione chiara sulla sua evoluzione.

Le architetture sono descritte tramite diagrammi a blocchi, i quali presentano una visuale sulla loro struttura e le relative relazioni. Per la specifica architetture useremo il linguaggio di modellazione unificato: **UML**, trattato nelle sezioni successive.

Naturalmente, prima di pensare alle componenti è necessario definire la struttura ad alto livello. Già in questa fase è possibile farsi un'idea degli eventuali costi di sviluppo, in termini di risorse e tempo, con lo scopo di definire un budget più realistico. Ciò fornisce un notevole aiuto durante i colloqui con gli stakeholder, così che loro possano esprimere le loro preferenze in merito alle caratteristiche del sistema. Il design architetture, infatti, impatta principalmente i seguenti campi:

- **Prestazioni:** Se lo stakeholder ritiene le performance del sistema la caratteristica più importante, sarà necessario spendere più tempo sull'ottimizzazione delle operazioni critiche.
- **Protezione:** Quando la protezione è un requisito critico, è comune utilizzare un'architettura a strati, al cui core sono presenti le risorse più importanti.
- **Sicurezza:** Se la sicurezza è importante, bisognerà localizzare le operazioni ad essa relative in un singolo componente o in pochi sottosistemi, con lo scopo di ridurre costi e problemi di convalida. Rende inoltre possibile fornire sistemi di protezione che possono bloccare il software in caso di guasto.
- **Disponibilità:** Se è richiesto che un sistema debba essere sempre disponibile, bisognerà progettare un'architettura con componenti ridondanti e meccanismi per la gestione di guasti, sicché si possa sostituire facilmente la componente problematica e allo stesso tempo mantenere il sistema attivo.
- **Mantenibilità:** Se è previsto un software dal lungo ciclo vitale bisognerà renderlo modulare per facilitare il lavoro del team. Saranno quindi presenti componenti piccole e con meno dipendenze possibili.

Non esiste il design architetture perfetto, è necessario scegliere quale modello si adatta al contesto degli stakeholder. Questo perché alcune architetture presentano conflitti; per esempio, risulta difficile creare un sistema le cui priorità sono prestazioni e protezione, perché, se si sceglie un design stratificato, per le operazioni critiche sarà presente un notevole overhead in quanto presenti al core, mentre le richieste sono eseguite al livello più esterno. È quindi verosimile cercare un compromesso tra i vari stili, piuttosto che adattarne uno solo.

È quindi estremamente importante conoscere più stili architetture per semplificare il lavoro di design e banalmente avere più idee riguardo all'implementazione del software per il dominio applicativo richiesto. Presentiamo quindi gli stili più utilizzati in ambito lavorativo:

- **Modello stratificato:** Organizza il sistema in un insieme di livelli; poggia su dei servizi core nascosti agli utenti, sui quali sono stratificati i livelli superiori. Ogni livello usa i servizi di quello inferiore e la comunicazione avviene nella maggior parte dei casi tramite chiamata a funzione.

Si tratta di un'architettura spesso utilizzata per la scrittura di sistemi operativi. Il vantaggio principale risiede nella sua modifica: se si lavora sull'interfaccia di un livello, si influenza esclusivamente quello adiacente, tuttavia, come menzionato prima, è un modello che può risultare inefficiente a causa dell'alto numero di chiamate fra i livelli.

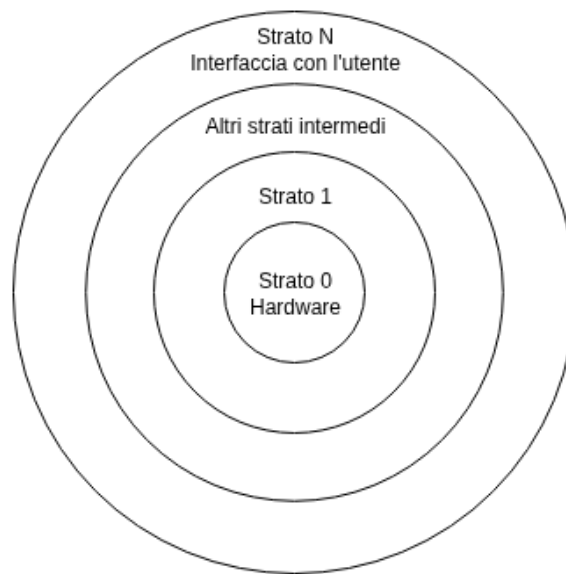


Figura 3.1: Modello stratificato

- **Modello repository:** Prevede un componente centralizzato contenente la memoria dei dati e componenti periferici che attuano operazioni di lettura e scrittura in questa memoria. Le periferiche non interagiscono tra loro e parlano esclusivamente con la componente centrale. È adatto ad applicazioni in cui i dati sono prodotti da un sottosistema e usati da un altro, come i case tool, strumenti che aiutano l'ingegneria del software durante il suo ciclo di vita.

Si tratta di un modo efficiente di condividere grandi quantità di dati con una gestione centralizzata del backup e della sicurezza. I sottosistemi dovranno tuttavia accordarsi sulla struttura dei dati per il repository; si tratta di un'evoluzione complessa e costosa, che rende difficile distribuire il software in maniera efficiente.

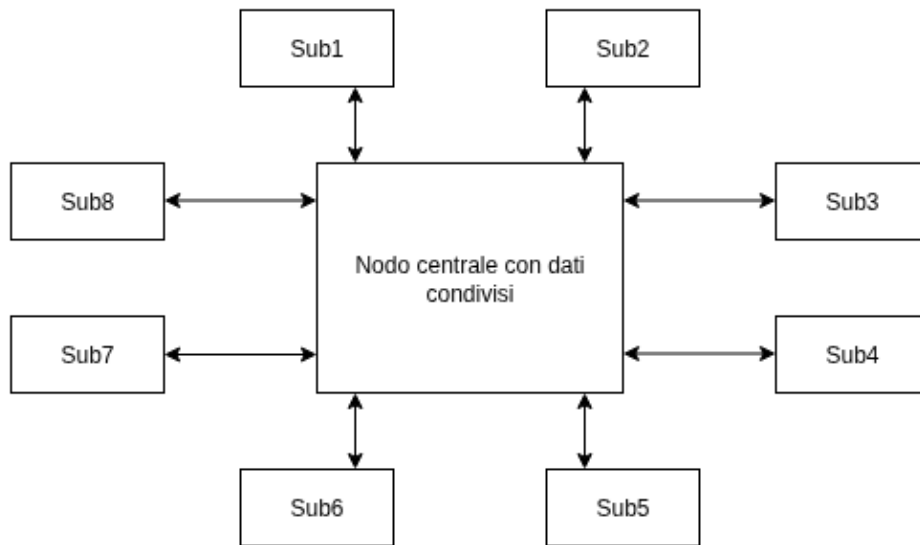


Figura 3.2: Modello repository

- **Modello client-server:** Presenta due entità fondamentali; il client, richiedente le informazioni, e il server, che le mantiene e fornisce. La comunicazione tra di loro avviene tramite una rete e le periferiche sono fornite da più server che svolgono appositi job a loro dedicati.

Si tratta di un modello usato normalmente dalle interfacce dei browser web; spostando la computazione tra i vari server si migliorano le prestazioni, il riuso e la manutenibilità. Tuttavia è un sistema distribuito, manca un modello di dati condiviso e il loro scambio può risultare inefficiente per questo. Non è semplice gestire il tutto perché possono presentarsi anche delle ridondanze e alcuni server potrebbero essere proprietari di un'azienda, quindi al di fuori del proprio controllo.

Esistono due evoluzioni di questo modello; la prima, **a due livelli**, presenta tre componenti software di **interfaccia utente**, **gestione dei processi e logica** e **gestione del database**, distribuiti sui due strati di client e server. La seconda, presenta **tre livelli**: uno di **presentazione**, un secondo per la **logica** ed il terzo per la **gestione dei dati**. Il client utilizza esclusivamente l'interfaccia utente, mentre gli altri componenti sono tutti gestiti dal server.

- **Modello peer to peer:** Questa architettura vede ogni componente svolgere il ruolo di client e server mentre esegue il proprio processo. Ogni elemento ha infatti un'interfaccia che specifica i servizi offerti. La comunicazione si basa quindi sulla richiesta e offerta dei servizi.

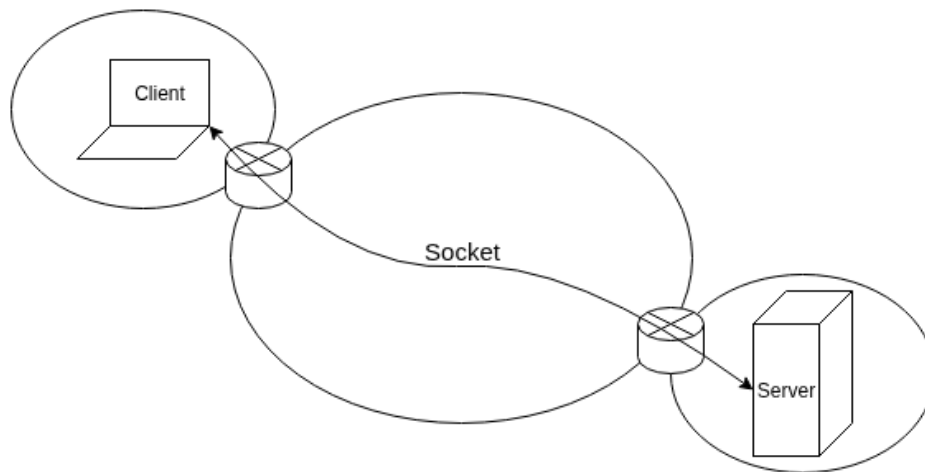


Figura 3.3: Modello client-server

I sistemi p2p scalano bene grazie alla facilità di inserimento dei nodi nuovi e sono tolleranti ai guasti, perché il "perno" sul quale si regge la struttura è decentralizzato.

- **Modello pipe and filter:** Presenta due entità fondamentali; i filtri, che effettuano trasformazioni sui dati, e i connettori che lo trasmettono ai vari filtri. Si tratta di un sistema che supporta l'esecuzione in parallelo e il riuso delle informazioni. Inoltre, ha un workflow intuitivo, rendendo semplice aggiungere e implementare trasformazioni. Tuttavia, non è adatto a software interattivi, ed è infatti usato spesso nello sviluppo di sistemi batch. Un esempio di questo sistema è il comando `grep` della bash UNIX.

3.2 Modelli di controllo

Per quanto riguarda il design architetturale, sono presenti anche degli **stili di controllo** del workflow per decidere come gestire il funzionamento del sistema. Ne esistono due principali:

- **Centralizzato:** Un sottosistema ha la responsabilità generale del controllo; avvia e termina gli altri componenti del software.
- **Basato su eventi:** Un sottosistema può rispondere ad eventi generati da altre entità, non esiste quindi un vero e proprio controllore.

L'importanza di conoscere i modelli di controllo risiede nello stesso contesto dei pattern architetturali: la definizione del "come fare" non è completa se si fornisce esclusivamente

una visuale astratta del sistema, ragion per cui andiamo nel dettaglio di come sarà gestito il workflow del software:

- **Modello call-return:** Presuppone l'esistenza di un sottosistema di controllo (**Main**) che attiva e cede il controllo ad altri componenti. Decide inoltre l'ordine di esecuzione e, al termine dell'esecuzione della sottocomponente, riprende il controllo del flusso. Si tratta di un modello applicabile esclusivamente a sistemi sequenziali.
- **Modello manager:** Pattern usato quando i sottosistemi possono lavorare in parallelo. Una componente **manager** controlla avvio, terminazione e coordinamento degli altri processi di sistema. Itera continuamente interrogando interfaccia utente, output ed eventuali sensori per identificare eventi o cambi di stato. Esempi di questo modello sono i sistemi di allarme; il manager interPELLa costantemente le sottocomponenti, le quali comunicano l'eventuale verificarsi di un dato evento con il segnale acustico.
- **Modello broadcast:** Contrariamente a quanto visto nei precedenti modelli, un componente può annunciare, quindi fare un broadcast, l'esecuzione di uno o più eventi. La trasmissione sarà inviata a tutte le altre entità, ognuna capace di gestirlo, e quelle interessate all'ascolto eseguiranno la procedura associata all'evento. Per questa ragione, un modello tale è chiamato anche **event-driven**.

I sistemi di sorveglianza utilizzano questo modello, poiché richiedono una costante allerta per gli eventi, la cui segnalazione deve essere comunicata a tutte le apparecchiature in ascolto. È facilmente mantenibile perché risulta semplice aggiungere e rimuovere entità. Tuttavia non è sicuro che l'evento venga gestito da una componente; come conseguenza diretta si potrebbero anche perdere dati importanti durante la trasmissione non ascoltata.

- **Modello a microservizi:** Approccio per lo sviluppo di una singola applicazione come una suite di piccoli servizi, ognuno eseguito nel proprio ambiente e comunicante con gli altri mediante meccanismi semplici, come una API di risorse HTTP. È un modello orientato ai servizi offerti dal software nettamente più mantenibile rispetto ai precedenti, poiché le varie componenti hanno dimensioni ridotte e sono indipendenti l'una dall'altra. Ogni microservizio realizza una funzionalità, possiede le proprie risorse ed è coordinato con gli altri.

Un'altra caratteristica di questo modello è che le varie componenti possono essere scritte in linguaggi di programmazione diversi, perché sono collegati da un gateway¹ per la loro coordinazione.

¹API Gateway. Si tratta di un design pattern.

Arrivati a questa fase del lavoro, si suppone che il design di alto livello, quindi delle specifiche del software e le sue relative interfacce, abbia avuto un riscontro positivo, consentendoci di andare più nel dettaglio e iniziare la **scrittura delle componenti**. Si determinano le specifiche dettagliate scrivendo interfacce dei singoli elementi che compongono il software, quindi i loro campi, metodi e relazioni necessari per svolgere la propria funzione.

Un primo metodo di design per la scrittura di componenti è il **design by contract**: una progettazione che implementa un **contratto** tra due entità, il quale deve specificare gli obblighi di entrambe per garantire il loro corretto funzionamento. Si crea un contratto per ogni componente del sistema prima che sia codificata ed è composto da più elementi:

- **Pre-condizioni**: Espressioni a valori booleani che rappresenta le aspettative sullo stato del programma prima dell'esecuzione di un'operazione.
- **Post-condizioni**: Espressioni a valori booleani riguardanti lo stato del software dopo l'esecuzione di un'operazione.
- **Invariante di classe**: Condizione che ogni oggetto della classe deve soddisfare in ogni momento in cui è possibile svolgere un'operazione.

Nei contratti si definiscono quindi responsabilità e condizioni da rispettare delle componenti coinvolte; quando queste sono violate, nella pratica viene lanciata un'eccezione. Si definiscono infatti due entità: un **client**, il cui compito è garantire la validità delle pre-condizioni, e un **fornitore**, che garantisce la correttezza delle post-condizioni in base alla validità di quanto passato dal client.

Questa divisione dei compiti ben specificata è molto utile per vari motivi: in primo luogo, facilita notevolmente il **debugging**, poiché grazie ai ruoli è possibile ricondursi alla componente che ha lanciato l'eccezione. Se non valgono le pre-condizioni, la colpa sarà del client, mentre se sono violate le post-condizioni il problema sarà situato nel fornitore. In secondo luogo, guida il team nell'implementazione e mantenimento del software, perché presuppone una chiara definizione dei compiti nel contratto, semplificando anche la scrittura degli unit test.

In generale, scrivere componenti con il metodo design by contract è molto importante, poiché senza una chiara definizione, si potrebbero effettuare controlli ripetuti delle condizioni in parti diverse del codice, danneggiando le prestazioni con le ridondanze.

3.3 Principi di progettazione

Ottenute le specifiche delle singole componenti, si passa alla fase di **design** delle **strutture dati** da implementare e **algoritmi** da utilizzare. Si tratta di un'attività lasciata in gran parte agli sviluppatori e presenta alcune sottofasi:

1. Analisi del design della classe da scrivere.
2. Selezione o definizione dell'algoritmo con la relativa notazione.
3. Scrittura dell'algoritmo mediante **step-wise refinement**.
4. Uso dei metodi formali per dimostrare la correttezza del codice.

Per la notazione degli algoritmi abbiamo a disposizione diversi strumenti, i quali possono essere **testuali**, come lo pseudocodice, oppure **visuali**, come diagrammi UML. Prendiamo ora un esempio pratico su cui utilizzare lo step-wise refinement: si inizia con la lettura delle specifiche e si definiscono le dinamiche dell'algoritmo, come i ruoli di client e fornitore insieme ai parametri dei metodi.

Dopodiché si passa ad una scrittura più profonda definendo il workflow dei singoli metodi, fino a quando non si è completato l'algoritmo. Lo scopo è ottenere una forma simile alla seguente:

Algorithm 1 getMax(a, b, c)

Require: $a: \text{int}, b: \text{int}, c: \text{int}$
 $\text{max} \leftarrow \text{calcMax}(a, b, c)$
 $\text{min} \leftarrow \text{calcMin}(a, b, c)$
 $\text{res} \leftarrow \text{max} - \text{min}$
return res

Quello del design è un processo creativo, ma richiede anche dei **principi di progettazione** per garantire uno standard e guidare lo sviluppo verso il raggiungimento di obiettivi di qualità. Ne esistono molteplici e portano a produrre software mantenibile, comprensibile, semplice da testare, riutilizzabile, portabile e chiaro.

Un primo modello è dato dai **moduli**: entità software identificate da un nome che contiene e fornisce servizi. Presenta un'interfaccia, la parte esternamente visibile, e la parte interna, composta da strutture dati e il codice. I moduli possono interagire fra loro e includerne altri (creando dipendenze); per esempio, in un programma C è possibile definire più file header da includere nel codice.

Un'altra idea è l'**astrazione**, la quale consente di concentrarsi su un problema ad un determinato livello senza perdersi in dettagli di quelli sottostanti. Aiuta a ridurre la complessità e permette di descrivere il comportamento dei moduli più facilmente. Inoltre, nasconde informazioni che gli altri livelli non devono conoscere. Ne esistono diverse forme:

- **Funzionale:** Definisce una funzionalità indipendentemente dall'algoritmo che la implementa.

- **Di dati:** Definisce un tipo di dato in base alle operazioni che si attuano su di esso senza definire una struttura concreta.
- **Di controllo:** Definisce un meccanismo di controllo senza mostrare i dettagli interni.

La terza idea è data dalla **decomposizione**, quindi la partizione di un problema complesso. Conosciuto anche come Divide-et-Impera, consente una più semplice gestione dei problemi in quanto di dimensione ridotta. Impiegando questo principio, tuttavia, sarà necessario spendere anche del tempo per l'integrazione di ogni modulo creato.

Una diretta conseguenza della decomposizione è la **modularità**, un principio che fornisce un criterio detto **separation of concerns** per la divisione del software: tenere separati gli aspetti che non si riguardano di un software. È infatti buona prassi mantenere separati in moduli le componenti di generazione dati (backend) da quelle necessarie alla presentazione (frontend), seguendo una linea guida per massimizzare la coesione e minimizzare l'accoppiamento. Un modulo si dice **coeso** se esegue un solo compito e usa le proprie risorse a disposizione, interagendo il meno possibile con altre entità.

L'**accoppiamento** è invece una metrica per il grado di dipendenza tra i moduli; quando è alto risulta più difficile apportare modifiche alle entità. Non è realistico avere moduli perfettamente indipendenti, ragion per cui è a discrezione del designer trovare un giusto compromesso. Generalmente, un buon accoppiamento è dato dalle chiamate di metodi di altri moduli; è inevitabile. Dei flussi evitabili sono invece quelli dove una componente modifica i dati di un'altra; sarebbe più corretto far gestire a un modulo le proprie risorse, eventualmente **nascondendole** agli altri.

I modificatori di accesso permettono di implementare il paradigma di **information hiding**; non tutte le informazioni devono essere pubbliche all'interno del software e quelle che non sono utili agli altri moduli dovrebbero essere rese private in quelle componenti dove sono necessarie. Se la tecnica è descritta tramite interfacce si ha come diretta conseguenza la creazione di moduli coesi.

È possibile rappresentare le dipendenze tramite un grafo; ciò aiuta a comprendere le modalità in cui i moduli sono accoppiati. Diciamo **fan-in** quando una dipendenza è entrante, mentre **fan-out** se è uscente. In particolare, un alto numero di fan-in indica un buon riuso del modulo in più contesti; al contrario, un alto numero di fan-out mostra un'eccessiva dipendenza e bisogna valutare se decomporre il modulo. Si richiede di trovare un compromesso.

Un'altra proprietà molto importante per i moduli è quella della **generalità**: impatta principalmente il grado di riutilizzabilità. È buona prassi cercare di rendere le componenti adattabili a più contesti se si ritiene possano essere utilizzate più volte. Un esempio è dato dagli algoritmi di ordinamento; se definiti in modo generale è possibile dar loro qualunque input e sapranno riportare un risultato corretto.

Infine, è impossibile non menzionare la tecnica **dry kiss**, ovvero **don't repeat yourself, keep it simple, stupid**. Bisogna scrivere codice il meno ripetitivo e quanto più chiaro possibile; se il workflow si "commenta da solo" leggendolo ed è presente una sola volta all'interno del programma, è stato fatto un buon lavoro.

3.4 Design Pattern

I **pattern di design** sono soluzioni standardizzate per dei problemi specifici di programmazione orientata agli oggetti, le quali costituiscono anche eventuali unità di riuso. Aiutano ad applicare i principi di buona progettazione, l'implementazione del design architetturale, ma possono anche essere utilizzati durante la modellazione e la codifica del software, facilitando la comunicazione tra sviluppatori. Altre unità di riuso disponibili sono

- **componenti riusabili (COTS)**: Parti di applicazioni di cui è fornita l'interfaccia, ma l'implementazione non è visibile.
- **Framework**: Insieme di classi e interfacce cooperanti che realizzano un design riutilizzabile e adattabile in più domini operativi.
- **Product lines**: Famiglie di prodotti sviluppati da uno stesso nucleo.
- **Librerie**: Insiemi di operazioni riutilizzabili in più contesti.

I design pattern si differenziano da queste unità perché rappresentano un concetto più astratto. Non è possibile applicarli direttamente, bensì è necessario adattarli al contesto operativo; inoltre, non concernono l'intera architettura del sistema, bensì solo alcune parti dove è ritenuto opportuno applicarli. Per esempio, in un sistema può capitare di avere la necessità di fornire uno spazio di dati condiviso, rendendo corretta l'applicazione di **Singleton**², ma ciò non significa che tutte le classi debbano necessariamente avere un solo oggetto. In ogni caso, i pattern elementari (GRASP) sono usati per l'assegnazione delle responsabilità nel software e costituiscono il fondamento per la progettazione di sistemi orientati agli oggetti. In particolare, distinguiamo tre sottocategorie:

- **Creazionali**: Definiscono modalità di istanziazione degli oggetti, come Abstract Factory.
- **Strutturali**: Definiscono la composizione di classi e oggetti, come Adapter e Facade.
- **Comportamentali**: Definiscono l'interazione tra gli oggetti e distribuiscono le varie responsabilità, come il Template Method, State e Observer.

²Design pattern che garantisce l'esistenza di un solo oggetto della classe in tal modo definita.

La descrizione di un design pattern presenta una struttura standardizzata, composta di quattro sezioni: il relativo **nome**, un **problema da risolvere**, la **soluzione** fornita dal pattern e le **conseguenze** che porta la sua applicazione. Vediamo alcuni esempi concreti.

Factory: Design pattern creazionale che fornisce un'interfaccia per la creazione di oggetti in una superclasse, ma consente alle eventuali sottoclassi di alterare il tipo di oggetti da creare.

Supponiamo di avere un sistema per la logistica. Una prima versione gestisce il trasporto solo mediante oggetti *Truck* e il grosso del codice è scritto al loro interno. Nel caso in cui si abbia la necessità di gestire trasporti con mezzi diversi, tuttavia, sarà necessario apportare modifiche all'intera codebase, perché la responsabilità non sarà più solo delle classi **Truck**.

La soluzione fornita da Factory suggerisce il rimpiazzo delle istanziazioni dirette degli oggetti, in questo contesto chiamati **prodotti**, con invocazioni ad un metodo ad hoc specifico per la loro creazione. Il vantaggio diretto di questo approccio è la possibilità di fare override del metodo in una sottoclasse ed eventualmente modificare la classe del prodotto da creare.

Un pattern simile è poi quello della **Abstract Factory**: pattern creazionale che consente di produrre famiglie di oggetti legati da una relazione senza specificare le loro classi concrete.

Immaginiamo di avere un sistema gestionale per un negozio di vendita di mobili. Il codice presenterà classi di famiglie dei vari prodotti, come *Chair*, *Sofa* e simili. Queste classi tuttavia presentano delle varianti ed è quindi necessario trovare un modo per creare oggetti concreti che rispecchino la caratteristica in esame. Inoltre, pensare di cambiare il codice esistente ad ogni nuovo tipo di prodotto è impensabile poiché estremamente dispendioso in termini di tempo e risorse. La soluzione Abstract Factory si attua in due fasi:

1. Dichiarazione di interfacce per ogni famiglia distinta di prodotti, quindi una per *Chair*, una per *Sofa*, etc, in tal modo che eventuali nuovi prodotti di quell'insieme seguano le stesse dinamiche.
2. Dichiarazione di una classe *AbstractFactory*, un'interfaccia con una lista di metodi creazionali per ogni prodotto facente parte di una determinata famiglia. Queste procedure devono ritornare necessariamente tipi di prodotto **astratti** rappresentati dalle interfacce definite prima. Per ogni variante di prodotto bisognerà creare una classe *Factory* concreta che implementa quella astratta e ritorna il prodotto concreto desiderato.

Il codice dovrà quindi lavorare con le fabbriche e i prodotti mediante le rispettive interfacce astratte. Ciò consente di cambiare il tipo di fabbrica che viene passato al richie-

dente, come anche la variante di prodotto che riceve, senza modificare il codice del client.

Per quanto riguarda i pattern strutturali, il primo degno di nota è **Adapter**³, il quale consente di far collaborare due oggetti le cui interfacce sarebbero altrimenti incompatibili.

Supponiamo di voler creare un'applicazione per monitorare il mercato azionario, la quale fa un download dei dati da più fonti per esportarlo in formato XML e mostrarlo in modo pulito con un'apposita interfaccia utente. Dopo un pò, si contempla l'idea di integrare una libreria di terze parti, ma questa lavora esclusivamente con file JSON.

La soluzione di Adapter richiede la creazione di un oggetto speciale che converte le interfacce di un oggetto sicché un altro possa comprenderlo e collaborarci. Un adattatore nasconde quindi la complessità nativa con un'astrazione comprensibile dall'altra parte, generalmente incurante del fatto che sia un adattatore.

Un prossimo pattern strutturale è **Facade**, che fornisce un'interfaccia semplificata a una libreria, un framework, o più in generale qualunque altro insieme complesso di classi.

Supponiamo di avere la necessità di rendere il codice funzionale con un vasto insieme di oggetti appartenenti a un Framework. La soluzione della facciata richiede la creazione di un'interfaccia più semplice con funzionalità essenziali dell'intero set di classi. Per esempio, un'applicazione atta al download di video su YouTube potrebbe usare una libreria contenente anche funzioni di riproduzione dei file. Queste ultime, tuttavia, esulano dallo scopo del nostro software, e la facciata non le comprenderebbe in quanto tali.

Passando ai pattern comportamentali vediamo anzitutto il **Template method**, che definisce lo scheletro di un algoritmo in una classe, le cui sottoclassi faranno override su step specifici della procedura senza modificarne la struttura intera.

Supponiamo di avere un'applicazione per il data mining, il cui scopo è estrarre informazioni dai documenti forniti. La prima versione gestisce solo il formato DOC, ma nelle successive si migliora il sistema rendendolo capace di lavorare anche sui CSV e PDF. Tuttavia, è evidente che molto codice è simile o duplicato nelle classi specifiche per formato.

La soluzione del template suggerisce di spezzare l'algoritmo in una serie di passi definiti in metodi diversi, ognuno dei quali potrà essere overrideato e reso specifico per il formato desiderato, rimuovendo il codice duplicato in toto.

Observer è un altro pattern comportamentale che consente di definire un "meccanismo di iscrizione" al quale possono prendere parte più oggetti, i quali saranno avvisati su ogni evento che concerne l'oggetto di loro interesse.

Immaginiamo di avere due oggetti: un negozio e un cliente; quest'ultimo è interessato in un particolare brand di prodotti che sarà presto reso disponibile. Il cliente potrebbe

³Chiamato anche Wrapper.

visitare il negozio ogni giorno, ma è verosimile che compia giri a vuoto perché non ha la certezza della disponibilità. Per risolvere questo fastidio, il negozio potrebbe inviare molte mail ai clienti per aggiornarli sullo stato della disponibilità, ma a coloro che non sono interessati potrebbe risultare altrettanto fastidioso.

In questo contesto chiamiamo **publisher** l'oggetto di interesse con funzione di comunicare il suo stato e **iscritti** gli altri oggetti interessati ai cambiamenti del primo. La soluzione di Observer suggerisce di aggiungere un meccanismo di iscrizione al publisher sicché altri oggetti individuali possano iscriversi o disiscriversi dalla sua "newsletter". Il meccanismo è generalmente costruito con un array che salva i riferimenti agli iscritti e vari metodi pubblici che consentono di iscrivere o disiscrivere gli oggetti.

Terminiamo lo studio dei design pattern con **State**, il quale consente a un oggetto di alterare il suo comportamento quando cambia il suo stato interno. Si tratta di una strategia relativa all'implementazione delle macchine a stati finiti (FSM).

Supponiamo di avere un'applicazione che funge da text editor, la quale può lavorare su documenti che possono presentare tre stati diversi: bozza, moderazione e pubblicazione. A primo impatto si penserebbe di usare un costrutto condizionale per cambiare modalità, ma con l'aumentare dei diversi stati, aumenterebbe di molto la complessità del codice.

La soluzione proposta da State suggerisce che si creino nuove classi per ogni stato di un oggetto e si estraggano tutti i comportamenti specifici dello stato in esse. Quindi, al posto di implementare tutti i comportamenti in un singolo oggetto, detto **contesto**, lo si usa solo per salvare un riferimento a uno degli oggetti di stato che rappresenta quello corrente, al quale si delega ogni funzione relativa. È bene che ogni stato implementi la stessa interfaccia.

Capitolo 4

UML - Unified Modeling Language

4.1 Introduzione a UML

Unified Modeling Language (UML) è un linguaggio standardizzato per il supporto alla descrizione e alla progettazione dei sistemi software. Sebbene sia tendenzialmente utilizzato in contesti orientati agli oggetti, è possibile usufruirne anche per descrivere sistemi hardware o comprendere il dominio operativo nel quale si lavora.

Presenta lo standard **Object Management Group** (OMG) e unifica in una singola collezione le varie notazioni pre-esistenti, integrandole e rendendole orientate agli oggetti. Si tratta di un linguaggio per la **modellazione** usato spesso in ambito lavorativo da analisti e progettisti software. Non è possibile usarlo per la programmazione. I tre utilizzi principali di UML riguardano:

- **Abbozzi:** Aiutano la comunicazione e la discussione, presentando solo le informazioni più importanti. Nella pratica sono usati tool come editor grafici o più direttamente una lavagna.
- **Progetti dettagliati:** Forniscono un modello completo da implementare mediante un modellatore UML, come Papyrus o Visual Paradigm.
- **Linguaggi di programmazione:** Meno comune dei due precedenti, è possibile usare dei tool appositi come UniMod per ottenere un modello eseguibile direttamente dalla composizione UML.

Esistono inoltre tre principali prospettive nell'utilizzo di UML: in primis, quella **concettuale**, che fornisce un punto di partenza per lo sviluppo con la rappresentazione dei concetti del dominio operativo; in secundis, quella di **specificazione**, che vede descritte nel diagramma le interfacce del sistema senza dettagli implementativi; infine, la prospettiva di **implementazione** descrive gli elementi concreti del sistema software, come le classi e gli attori.

I sistemi in UML sono destritti tramite le **viste**, intese come un particolare aspetto del software da un punto di vista di un ruolo specifico. Lo scopo di un diagramma è la descrizione di una vista e il loro insieme compone il modello. Lo standard utilizzato in questo corso è UML 2.0, il quale contiene 13 tipi di diagrammi diversi divisi in due macro-categorie: **diagrammi strutturali**, i quali descrivono la struttura statica del sistema e **diagrammi comportamentali**, che rendono il comportamento dinamico.

UML è un linguaggio versatile; può essere utilizzato con qualsiasi processo nelle fasi di raccolta requisiti, design architetturale e delle componenti. Per esempio, nella prima fase è possibile usare un use case diagram per la descrizione dei requisiti funzionali del sistema da una vista dell'utente.

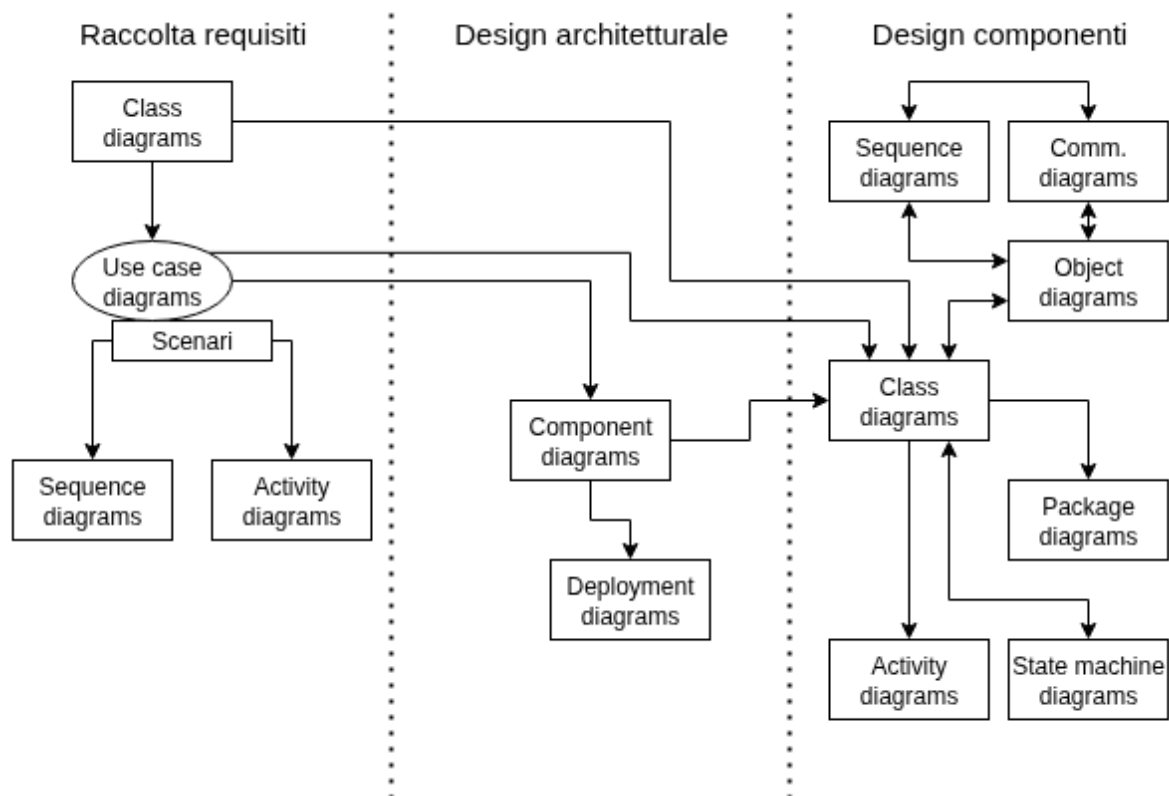


Figura 4.1: Panoramica diagrammi UML 2.0

Non sempre è sufficiente l'implementazione dei diagrammi UML nel workflow; esistono due soluzioni principali per essere più esaustivi: **non usare** UML oppure l'**estensione** del diagramma tramite **profili**, andando a sacrificare lo standard. Per quest'ultima soluzione è possibile specificare particolari domini applicativi o tipi di applicazioni; un profilo è costituito principalmente da:

- **Stereotipi:** Elementi aggiuntivi ottenuti modificando quelli già esistenti.
- **Vincoli aggiuntivi:** Ulteriori restrizioni rispetto a quelle standard presenti in UML.
- **Informazioni semantiche aggiuntive:** Per dare ulteriori delucidazioni in merito agli elementi.

Quello di estendere i diagrammi rendendoli **illegali** è un compromesso utilizzato spesso in ambito lavorativo. Ciò fa capire quanto sia fondamentale la flessibilità ai contesti rispetto all'attenersi esclusivamente agli standard e ai manuali. Lo sviluppo software è un processo creativo. Divertiti.

4.2 Diagrammi strutturali

I **diagrammi di classe** definiscono le classi e relativi attributi, operazioni e relazioni con altre entità, come associazioni, aggregazioni, composizioni, specializzazioni, generalizzazioni e dipendenze. Modellano quindi la parte strutturale (o statica) di un sistema software.

Il concetto di classe in UML è identico a quello presentato nel paradigma di orientamento agli oggetti; vengono rappresentate da un rettangolo diviso in più compartimenti, i quali contengono nome, attributi e operazioni rispettivamente. Per quanto riguarda gli **attributi** presentano la segnatura:

visibilità nome: tipo [molteplicità] = default{proprietà}

Dove default è il valore base dell'attributo di un oggetto appena creato e proprietà definisce vincoli come sola lettura. Il nome è l'unica parte obbligatoria da indicare, mentre le altre sono facoltative. Presentano inoltre quattro simboli corrispondenti ai modificatori di accesso, validi anche per le operazioni, discusse più avanti:

- **Public:** +
- **Private:** -
- **Protected:** #
- **Package private:** ~

Anche la **molteplicità** è un dato che può presentare più valori e corrisponde a un quantitativo degli attributi, come per esempio le dimensioni di una lista concatenata. Il valore di **default** è 1 e indica che nel software esiste una sola classe di questa fattura. Alternativamente è possibile usare 0..1 per dire **al più uno**, 0..* per **nessuno o numero imprecisato**, 1..* per **almeno uno**, oppure per generalizzare, usare n, m , definendo i loro valori. Le **operazioni** invece presentano la segnatura:

visibilità nome(parametri): tipo-ritornato{proprietà}

Le **operazioni** presentano una segnatura simile a quella degli attributi, ma sono capaci di ricevere dei parametri all'interno delle parentesi tonde. Sarà necessario dar loro una direzione con una freccia per permettere il collegamento fra le classi. Inoltre, essendo che è comune prassi la scrittura di metodi getter e setter, generalmente sono omessi dallo spazio apposito.

In UML esistono due tipi di operazioni: le **query**, le quali ottengono un valore da un oggetto senza effetti secondari, e i **modificatori**, che modificano lo stato dell'oggetto su cui sono chiamate.

Per una migliore specifica, si fornisce il concetto di **datatype**, equivalente ad un valore definito dall'utente; possono essere una data o un valore in Euro, per esempio. Si rappresentano come classi alle quali è aggiunto lo stereotipo <<datatype>>.

Naturalmente, le classi possono essere messe in relazione. Il meccanismo è ottenuto tramite **associazioni**, annotate tramite frecce che mostrano la direzione con la quale deve essere letta la relazione. Per esempio, se una classe "Person" ha una freccia uscente che entra nella classe "car", significa che è possibile ottenere informazioni riguardo l'automobile a partire dalla persona.

Inoltre, è possibile porre una molteplicità alle associazioni, mostrando quanti oggetti possono esistere durante l'esecuzione del software con metriche identiche a quelle discusse in precedenza. Attributi e associazioni sono due modi per rappresentare la stessa cosa, ma per convenzione si usano i primi per campi di tipi primitivi e datatype, mentre le seconde se il tipo è definito da una classe.

Con lo scopo di rendere ulteriormente comprensibile un class diagram è possibile integrarlo tramite gli **object diagram**, rappresentanti le istanze di una classe, spiegando anche la relazione tra classe e oggetti mediante l'operazione <<istanza>>. È possibile segnalare operazioni e attributi statici sottolineandoli.

Altre casistiche possibili nella scrittura dei diagrammi di classe o di oggetti sono le **associazioni riflessive**, dove una classe ha un'associazione con sé stessa, le **classi associative**, dove le proprietà appartengono all'associazione piuttosto che alle classi, le **associazioni bidirezionali**, che rendono le informazioni reperibili da entrambe le classi coinvolte e richiedono una sincronizzazione quando vengono istanziate, le **aggregazioni**¹, con le quali è possibile rappresentare la relazione fra un insieme e una delle sue parti con un rombo alla base della freccia, le **composizioni**, forme più rigorose delle aggregazioni, dove se è deallocato il composto, allo stesso modo seguiranno le sue parti.

Abbiamo infine tre concetti molto più vicini alla programmazione orientata agli oggetti, come la **generalizzazione ed ereditarietà**; un meccanismo grazie al quale elementi specializzati incorporano struttura e comportamento di elementi più generali. Inoltre,

¹Consigliabile utilizzarle solo a livello concettuale. In ambito concreto possono confondersi con le associazioni.

è possibile segnalare le **dipendenze** tra le classi con una freccia tratteggiata; in loro presenza, la modifica di una classe ne comporta una anche nell'altra. Esistono degli stereotipi appositi, come `<< use >>`, `<< call >>` e `<< create >>`.

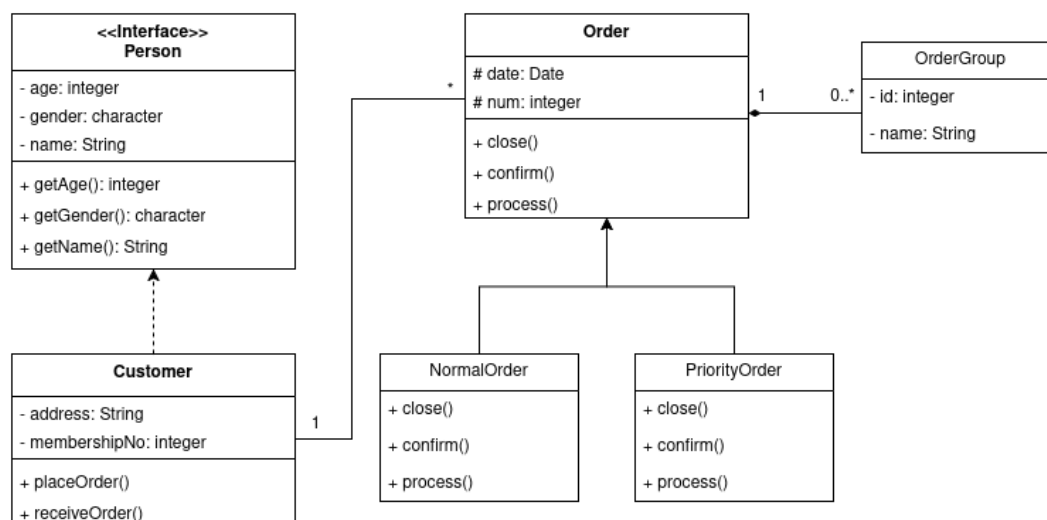


Figura 4.2: Esempio class diagram

Introduciamo ora il concetto di **interfaccia**, inteso come l'insieme delle operazioni visibili all'esterno degli oggetti, istanze di quella classe. Si tratta di un'entità priva di implementazione simile a una classe, definita con classi concrete che fanno uso della sua struttura. La relazione è generalmente indicata con frecce tratteggiate verso la classe interfaccia, la quale conterrà lo stereotipo `<<interface>>`.

In alternativa, è possibile usare la notazione **lollipop**, dove due classi sono collegate con una linea retta, al cui centro è presente un cerchio (lollipop) che indica il nome dell'interfaccia richiesta, e un semicerchio (socket), che indica la classe richiedente.

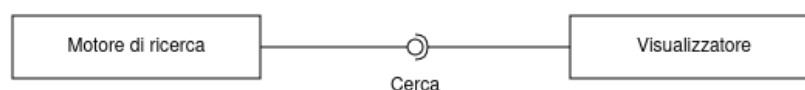


Figura 4.3: Esempio interfaccia

Il vantaggio principale che si ottiene utilizzando interfacce è una diminuzione dell'accoppiamento tra classi; se durante la scrittura del codice si nota che più classi utilizzano meccanismi simili, potrebbe essere corretto astrarli con un'interfaccia e definire i metodi nelle classi concrete².

²Questo è un esempio di un design pattern chiamato Strategy.

In generale, le associazioni, gli attributi e le generalizzazioni sono costrutti base per esprimere vincoli, ma non sempre bastano a rappresentarli correttamente. Per questa ragione UML permette di specificarne ulteriori utilizzando il linguaggio naturale o un **Object Constraint Language**³ (OCL), basato sulla logica del primo ordine.

4.3 Diagrammi di interazione

I **diagrammi di interazione** sono un sottoinsieme di quelli che descrivono le modalità di interazione tra attori e sistema in termini di scambio di messaggi, i quali possono essere **sincroni**, dove il mittente si pone in attesa del risultato, **asincroni**, quando può continuare l'esecuzione concorrentemente all'elaborazione del risultato, e **trovati**, per i quali il partecipante che lo genera è omesso.

Il primo diagramma trattato è il **sequence diagram**, la cui notazione presenta due attori (le classi) principali, detti partecipanti, e indicati da tre possibili segnature:

- nomeClasse: Oggetto di nome "nomeClasse" il cui tipo non è specificato.
- :nomeClasse: Istanza anonima della classe "nomeClasse".
- s:nomeClasse: Oggetto di nome "s" della classe "nomeClasse".

Il contesto dei diagrammi di sequenza è descritto ponendo i partecipanti in cima al diagramma, con linee tratteggiate verso il basso che indicano il flusso del tempo. Su queste linee saranno disegnati dei rettangoli per indicare il tempo di esecuzione; da questi rettangoli è possibile segnare lo scambio di messaggi tramite frecce. In particolare, se una freccia ha riga normale, è un'invocazione, mentre se è tratteggiata, è la sua risposta. Sulle frecce dei messaggi è possibile specificarne anche i parametri, con la segnatura:

returnValue = message(params): returnType

I messaggi possono essere usati anche per la creazione di oggetti; in tal caso si indica con una freccia tratteggiata e il metodo new al rettangolo nuovo contenente il nome dell'oggetto. Se questo deve poi essere liberato, nella sua linea del tempo è sufficiente porre una croce per indicare il termine della sua esistenza.

In questo diagramma è possibile indicare condizioni e cicli con i **frame di interazione**, in particolare **Alt** per il costrutto if-else, **loop** per while, e **opt** per indicare attività opzionali. La condizione, detta **guardia**, è indicata tra parentesi quadre vicino al nome del frame. È inoltre possibile annidare i contenitori, ma è sconsigliato farlo poiché riduce la leggibilità del diagramma.

Un frame usato proprio come soluzione a questo problema è **ref**, usato per riferirsi a un metodo specifico, come per le chiamate a funzione. Esiste infine il frame **par**, che indica i metodi eseguiti in parallelo.

³È un linguaggio usato anche per la specifica di pre e post-condizioni

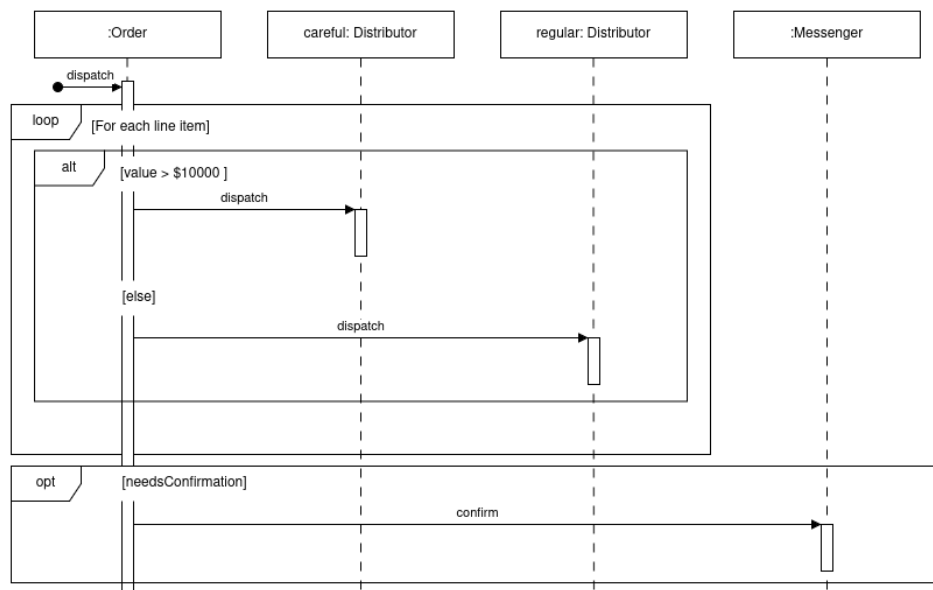


Figura 4.4: Esempio sequence diagram

I sequence diagram sono un ottimo strumento per visualizzare l'interazione tra gli oggetti da un punto di vista astratto, mostrando con chiarezza le relazioni tra di essi senza andare nel dettaglio concreto. Per questa ragione sono spesso utilizzati a livello di requisiti per spiegare gli use case e a livello di design per descrivere le interazioni tra le classi.

Un'altra opzione per la descrizione del comportamento sono i **communication diagram**, i quali hanno sintassi uguale a quella appena discussa, ma rappresentano i messaggi come etichette direzionali degli archi, numerati sequenzialmente.

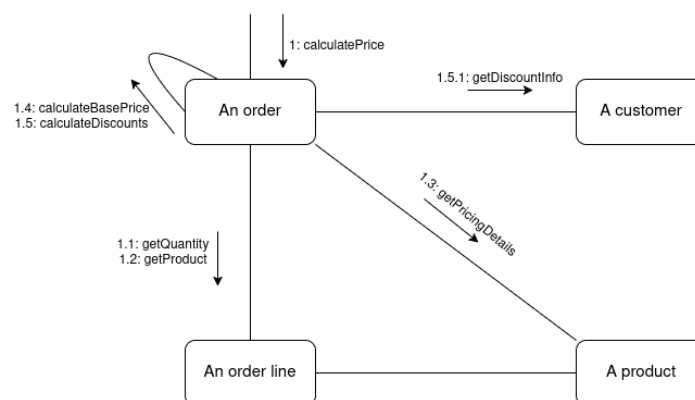


Figura 4.5: Esempio communication diagram

4.4 State Machine Diagram

Gli **state machine diagram** descrivono il comportamento di un componente, in termini di variazione del suo stato interno alla ricezione di un input di qualche tipo, inteso come un'istanza ottenibile durante la vita di un oggetto, per la quale sono soddisfatte determinate condizioni.

Rappresentiamo questo diagramma tramite nodi (stati) e archi (transizioni) e gli eventi ricevuti dal sistema sono salvati in una coda, dalla quale sono poi estratti uno alla volta. Presenta una semantica **run-to-completion**⁴ e deve rispettare un comportamento deterministico.

Nel diagramma sono presenti anche un singolo punto di inizio, indicato da un disco nero, ed uno o più stati finali, annotati con un disco nero bordato. Le transizioni indicate negli archi specificano poi le modalità di passaggio tra gli stati; indicano l'**evento** scatenante il passaggio di stato, la **guardia** che se verificata permette la transizione, e le **attività** compiute prima di cambiare stato, tipicamente espresse in linguaggio naturale. Questi parametri sono opzionali.

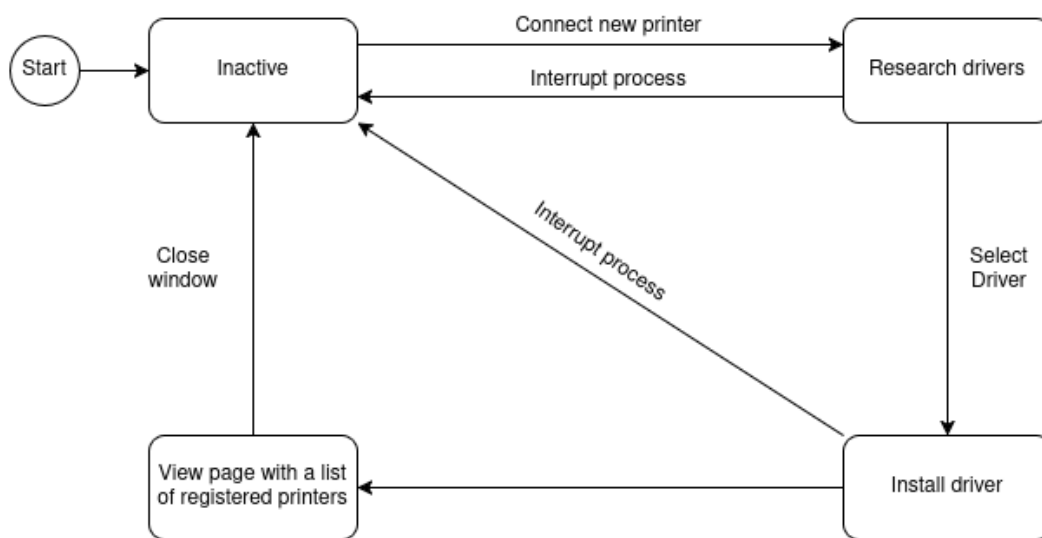


Figura 4.6: Esempio state machine diagram

Esistono più tipi di evento oltre alla ricezione di un messaggio; per esempio, quando una condizione passa da falsa a vera, diciamo che è un evento di **cambiamento**, oppure dopo un intervallo di tempo può accadere un evento **temporale**. Non è l'unico modo per gli stati di svolgere mansioni, tuttavia. Infatti, possono compiere delle **attività interne**

⁴Un elemento è estratto dalla coda esclusivamente quando il precedente è stato interamente processato.

annotabili nel diagramma in una partizione del loro rettangolo con una sintassi uguale a quella delle transizioni; in particolare, esistono anche attività speciali dette **entry** ed **exit**, le quali si eseguono all'entrata nello stato e alla sua uscita rispettivamente, come anche **do-activity**, eseguita mentre l'oggetto si trova in quello stato.

Inoltre, gli stati si differenziano in **semplici**, come quelli discussi finora, e **compositi**, che possono comprendere più automi a stati finiti. Nei diagrammi che mostrano l'intero sistema, questi saranno annotati con un'icona, un entry point e un exit point, per poi essere esplicitati nel dettaglio con un diagramma differente. Risultano molto utili per descrivere la concorrenza o comportamenti non deterministici.

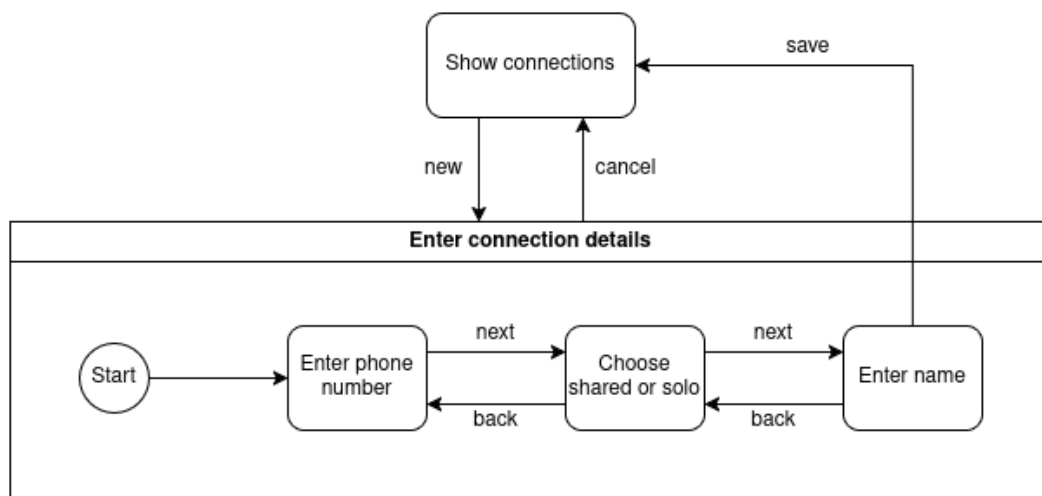


Figura 4.7: Esempio stato composito

Le macchine a stati si implementano generalmente in tre modi: con un costrutto switch, il design pattern State, oppure dando una state transition table a un tool per la traduzione in codice.

4.5 Activity Diagram

Gli **activity diagrams** descrivono le modalità di interazione relative a una o più entità, intese come operazioni di una classe, le classi stesse, componenti o attori. Definisce il flusso di azioni che accadono o devono accadere, in ordine, con una sintassi flowchart-like. Sono diagrammi versatili, adattabili a più contesti; sono utilizzati per modellare il flusso di lavoro di un business, di use case, oppure anche come alternativa allo pseudocodice.

Le attività sono definite come un flusso di azioni, singole componenti del diagramma collegate da frecce, dette **flussi**, e supportano l'elaborazione concorrente. Gli elementi del grafico si dicono **nodi** e ne esistono di tre tipi:

- **Azione:** Rappresentati con rettangoli dagli angoli smussati, specificano le unità di comportamento.
- **Oggetto:** Rappresentati con rettangoli classici, specificano gli oggetti usati come input/output.
- **Di controllo:** Comprendono nodo di inizio (disco nero), attività (disco nero bordato), flusso (disco vuoto con croce interna), parallelismo (riga spessa) e decisione (rombo) e specificano il flusso delle attività.

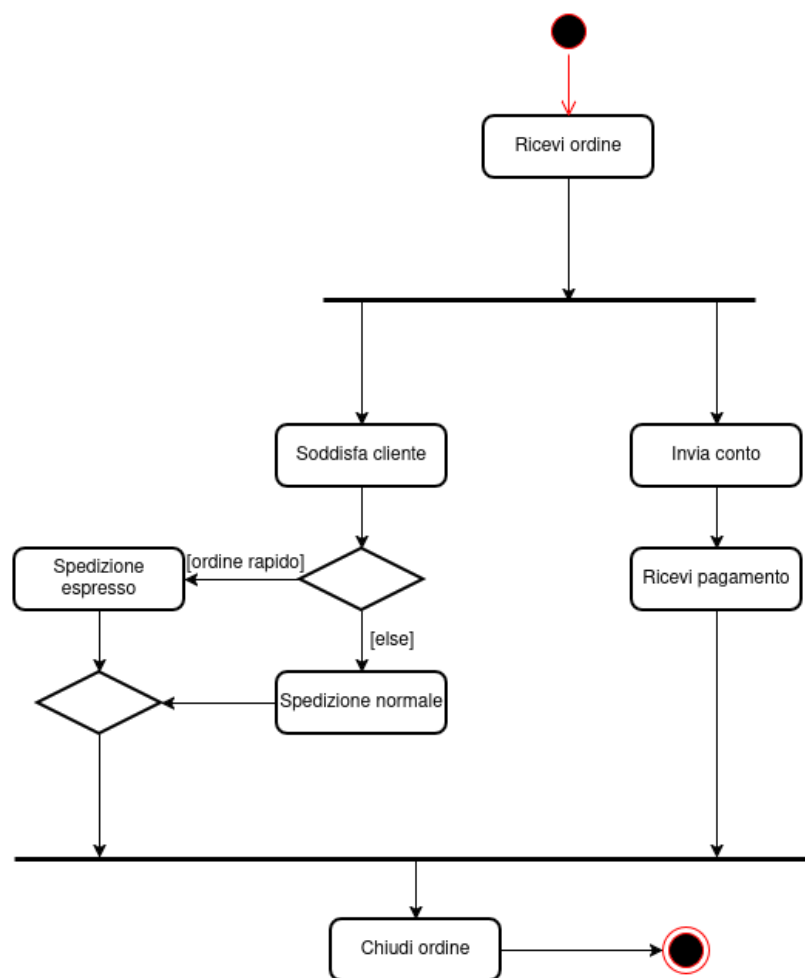


Figura 4.8: Esempio activity diagram

I nodi **azione** possono essere espressi in linguaggio informale, invocare altre attività segnalandolo con il simbolo rake, chiamare l'operazione di una classe con una sintassi

method reference simil-Java, oppure scrivere un frammento di pseudocodice. La posizione all'interno dei nodi di un flusso è indicata tramite **token**, il quale si propaga sul nodo connesso dal suo arco in uscita. La terminazione avviene, come per lo state machine diagram, quando il token raggiunge il nodo finale.

I nodi **decisione** hanno un input e vari output, direzionano il token ricevuto verso uno dei propri output in base alla guardia usata. La stessa forma sintattica prendono i nodi fusione, con la differenza che possono avere più input, ma un singolo output. Quelli di **fork** sono usati per eseguire operazioni in parallelo, mentre i **join** li riuniscono in un solo filo di esecuzione. Inoltre, se il nodo finale è raggiunto da un thread quando ne esistono altri attivi, terminerà il flusso di tutti gli altri.

I nodi **oggetti** modellano gli oggetti in input e in output delle azioni; è possibile esplicitarne anche uno stato per mostrare la loro evoluzione durante il flusso, ma, se esiste, devono essere coerenti con la macchina a stati ad esso associata. Per rendere il diagramma ulteriormente informativo si può partizionare il nodo oggetto mostrando l'esecutore delle azioni e una suddivisione delle attività ad esso associate.

Esistono inoltre alcuni nodi specializzati per la gestione di invio (non bloccante) e ricezione (bloccante) di eventi; questi ultimi in particolare sono attivi quando presentano token su tutti gli archi in entrata, se ne hanno, oppure durante l'intera vita dell'attività quando non ne hanno. Generano inoltre i token alla ricezione di eventi, sia temporali che non. Infine, è possibile indicare parametri specifici e **regioni interrompibili** per mostrare il comportamento del flusso in caso di interruzione forzata delle attività, come in caso di un'eccezione.

In genere, i diagrammi di attività sono utilizzati per la spiegazione e modellazione di casi d'uso. Questi devono seguire uno schema generale, composto da una breve descrizione, gli attori primari e la sequenza principale degli eventi, sul quale basare la stesura del diagramma con le relative diramazioni del flusso di lavoro.

4.6 Component, Package e Deployment Diagram

Capitolo 5

Evoluzione e mantenimento software

5.1 Refactoring

5.2 Unit testing